AGENT EVOLUTION · NOTEBOOK 6 OF 8

Version 6 — Middleware: a SQL Budget

A guide to `version6_agent_budget_middleware.ipynb`: what middleware is, where a tool wrapper sits between the model and the tool, and how `SQLBudgetMiddleware` counts and blocks `run_query` calls using agent state.

PA

Tutor: Prudhvi Akella
Lead Data & AI Engineer · 13+ years in data engineering and AI architecture

Stack: LangChain `create_agent` · `AgentMiddleware` · LangGraph state and `Command`

Tools: same 6 as Version 5 · Prompt: unchanged · New: `SQLBudgetMiddleware(max_sql_calls=3)`

1. What changes from Version 5
2. What middleware is
3. Where a tool wrapper sits
4. The wrapper pattern
5. Inside `SQLBudgetMiddleware`
6. The counter lives in state
7. Blocking a call
8. Sync and async hooks
9. Registering middleware
10. One run, step by step
11. Why code, not a prompt rule
12. What the budget does not do
13. The limitation, and what Version 7 changes
14. Key takeaways

1 What changes from Version 5

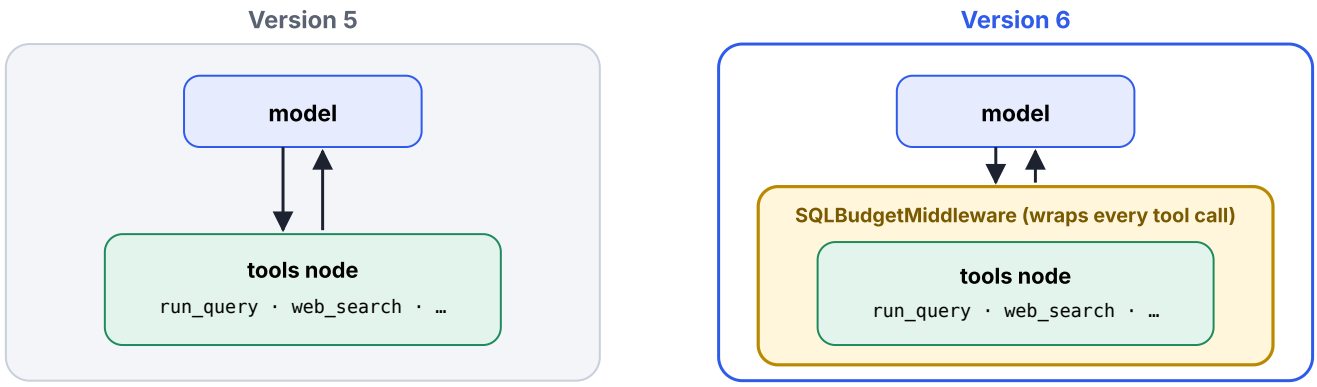


Figure 1. Same model, same tools, same prompt. A wrapper now sits around every tool call.

	Version 5	Version 6
Tools	6	same 6, unchanged
Prompt	Version 5 prompt	**unchanged**
`create_agent(...)`	no middleware	`middleware=[SQLBudgetMiddleware(max_sql_calls=3)]`
Agent state	`messages`	`messages` + `sql_call_count`

Limit on SQL runs	none	3 `run_query` executions per run

Nothing the model sees has changed: same tools, same prompt. Any difference in behaviour comes from the new code around the tools. That is why the prompt was frozen in Version 5.

2 What middleware is

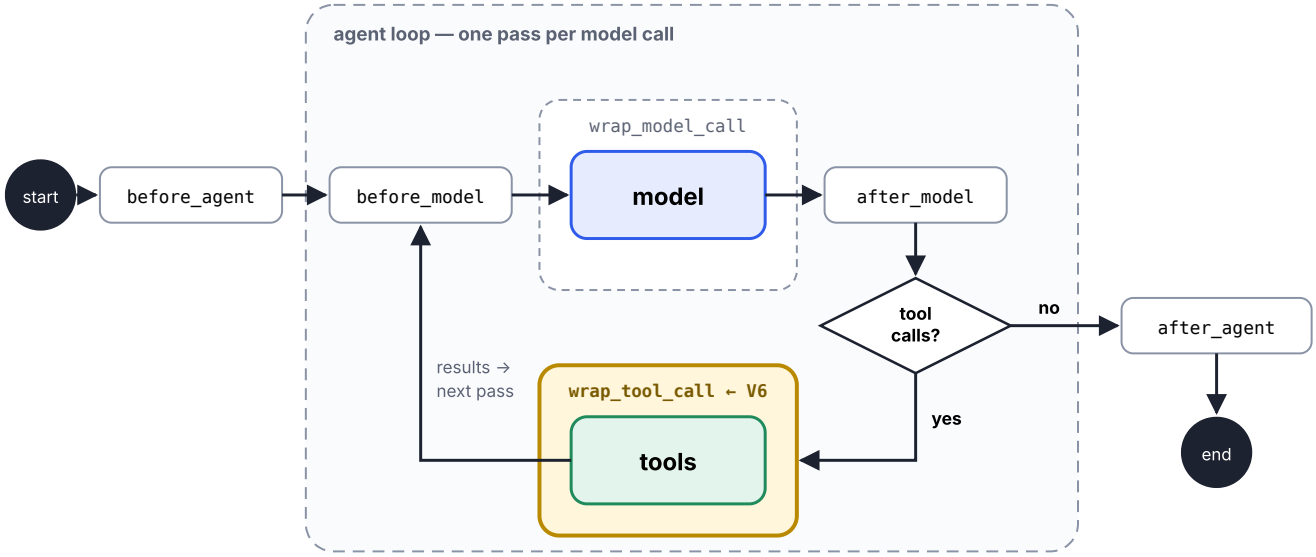


Figure 2. The agent loop with every middleware hook LangChain offers. Version 6 uses only `wrap_tool_call`.

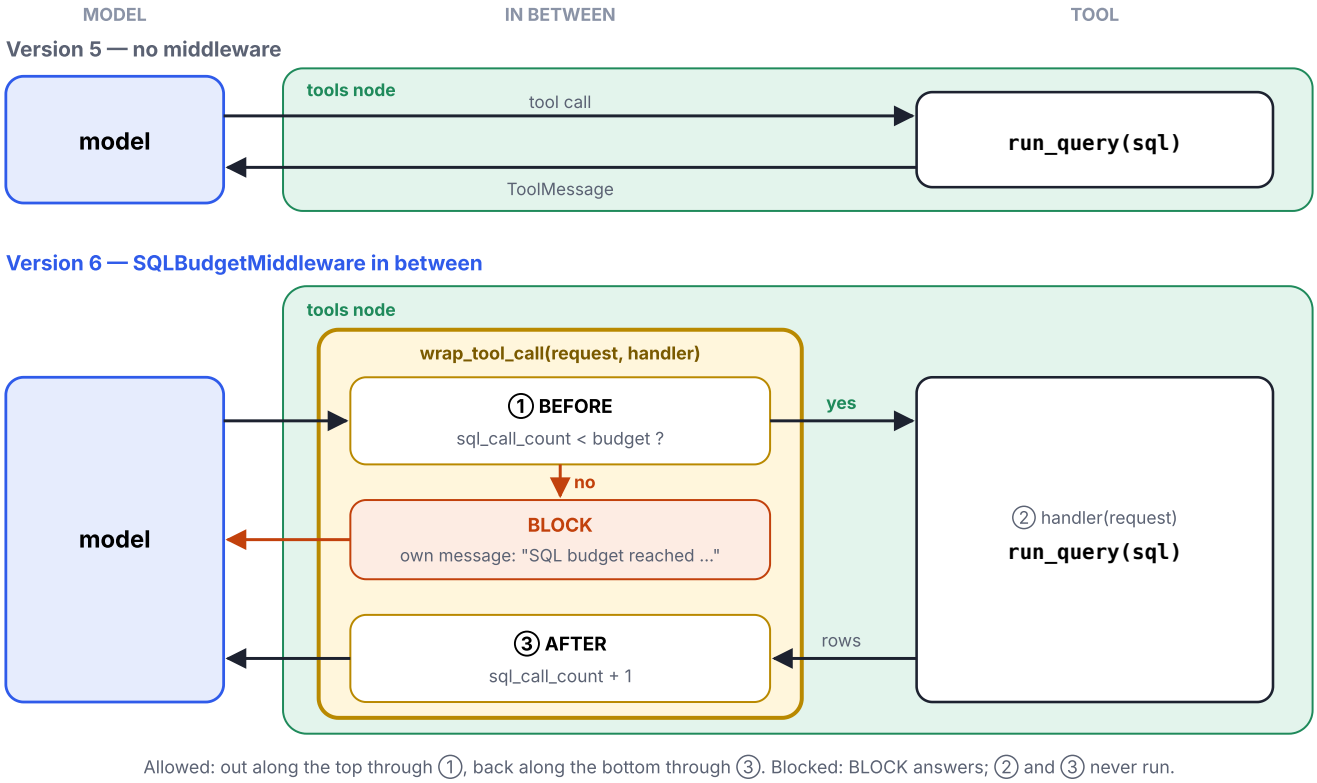
Middleware is code you attach to an agent that runs at fixed points in its loop. The model and the tools stay the same; middleware adds behaviour around them.

`AgentMiddleware` has six hook points. Each also has an async version with an `a` prefix (`abefore_model`, `awrap_tool_call`, ...).

Hook	Runs	Typical use
`before_agent`	once, when a run starts	load context, set up state
`before_model`	before every model call	trim or summarise messages
`wrap_model_call`	around every model call	retries, model fallback, caching
`after_model`	after every model reply	check or stop the reply
`wrap_tool_call`	**around every tool call**	**limit, block, or change tool calls** — Versions 6 and 7
`after_agent`	once, when a run ends	logging, clean-up

> A middleware class overrides only the hooks it needs. `SQLBudgetMiddleware` overrides `wrap_tool_call` (and its async twin) and nothing else.

3 Where a tool wrapper sits



Allowed: out along the top through ①, back along the bottom through ③. Blocked: BLOCK answers; ② and ③ never run.

Figure 3. The model's tool request no longer goes straight to the tool. It passes through the wrapper first, and the result passes back through it.

This is the key picture for Version 6. When the model asks for a tool, the `tools` node does not call the tool directly any more. It calls `SQLBudgetMiddleware.wrap_tool_call`, and hands it a function called `handler` that would run the tool.

1. **The model replies** with a tool call, e.g. `run_query(sql="SELECT …")`.
2. **The wrapper runs first.** It looks at the request and the current state, and decides.
3. **If allowed,** the wrapper calls `handler(request)`. Only now does `run_query` run.
4. **The result comes back through the wrapper,** which can add to it (here: one more count) before it goes back to the model.
5. **If not allowed,** the wrapper never calls `handler`. The tool does not run. The wrapper writes its own reply.

> The model cannot tell the wrapper is there. It asked for `run_query` and got a `ToolMessage` back, exactly as in Version 5. Only the content of that message can differ.

4 The wrapper pattern

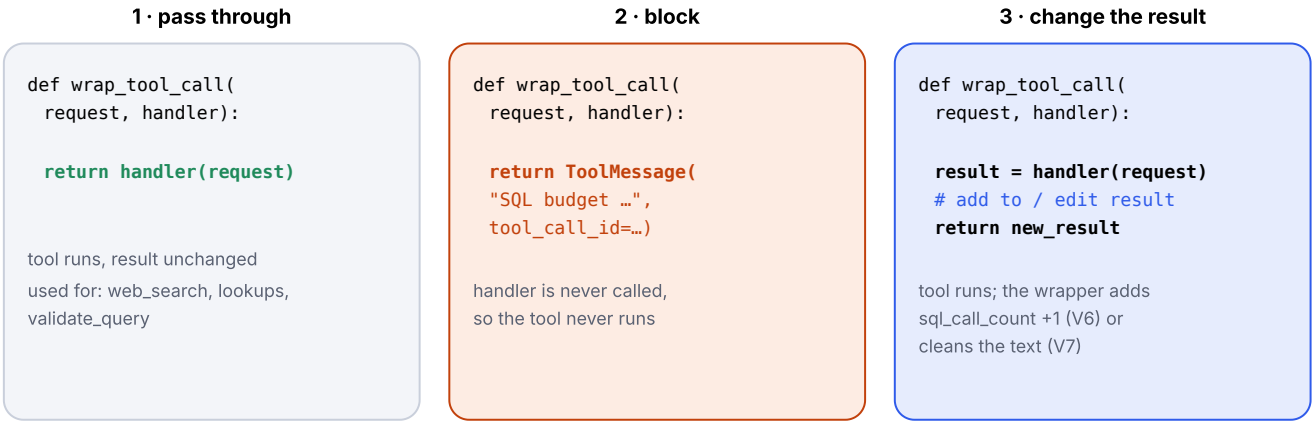


Figure 4. Everything a tool wrapper can do is one of these three moves. `SQLBudgetMiddleware` uses all three.

A tool wrapper is a function that receives two things and returns one:

Name	What it is
`request`	a `ToolCallRequest` with four fields: `tool_call` (name, args, id), `tool` (the tool object), `state` (the agent state right now) and `runtime`
`handler`	a function that runs the tool (or the next wrapper) when you call `handler(request)`
return value	a `ToolMessage`, or a `Command` that carries one plus state updates

Code before `handler(request)` runs before the tool. Code after it runs after the tool. Not calling `handler` at all means the tool does not run.

`SQLBudgetMiddleware` picks a move per call:

- any tool other than `run_query` → **pass through**;
- `run_query` with budget used up → **block**;
- `run_query` within budget → run it, then **change the result** by adding one to the counter.

5 Inside `SQLBudgetMiddleware`

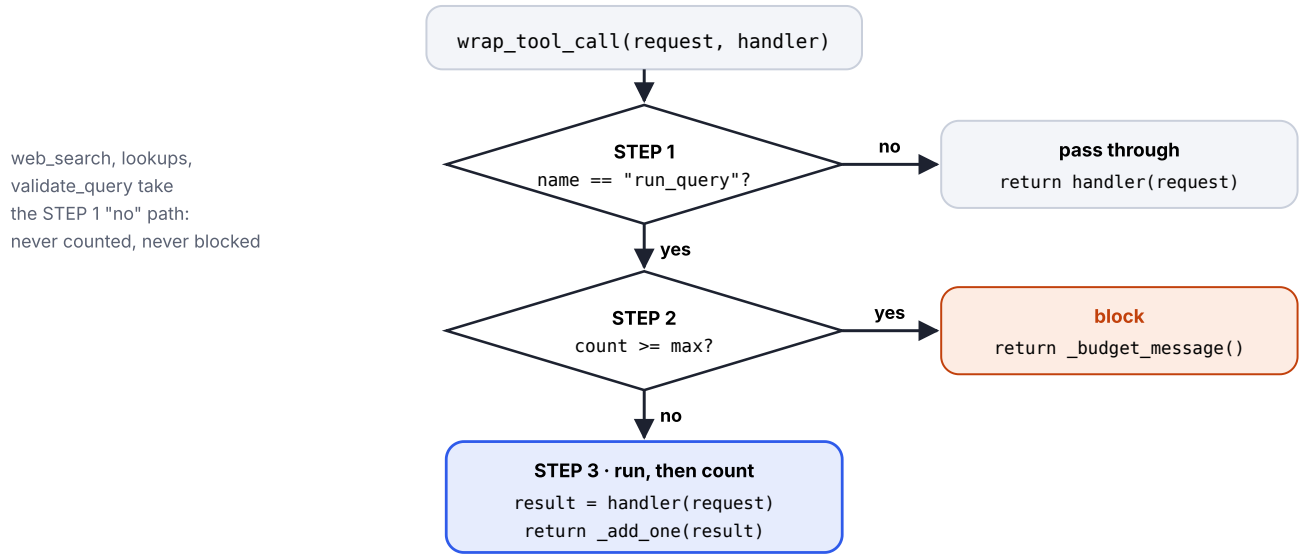


Figure 5. The three steps, matching the STEP markers in the notebook's code.

```
def wrap_tool_call(self, request, handler):
    if not self._is_sql_call(request):          # STEP 1  not run_query → pass through
        return handler(request)
    if self._budget_used_up(request):           # STEP 2  count >= max → block
        return self._budget_message(request)
    return self._add_one(handler(request))      # STEP 3  run it, then count it
```

Helper	What it does
`_is_sql_call`	`request.tool_call["name"] == "run_query"`
`_budget_used_up`	`request.state.get("sql_call_count", 0) >= self.max_sql_calls`
`_budget_message`	builds the “SQL budget reached” `ToolMessage` (section 7)
`_add_one`	wraps the result in a `Command` that also adds 1 to the counter (section 6)

The decision logic is in small helpers, so the sync and async hooks (section 8) can share it and stay identical.

6 The counter lives in state

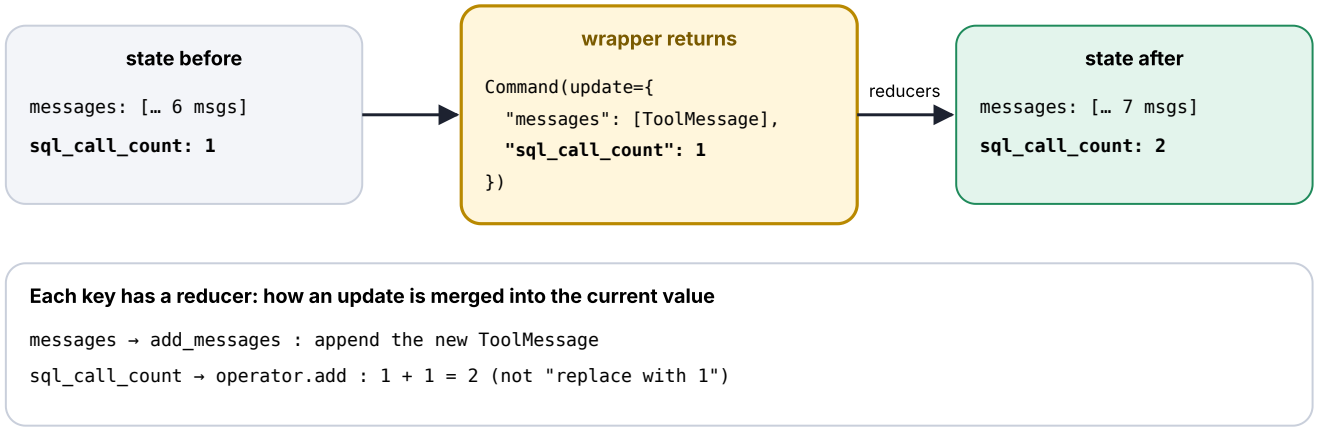


Figure 6. The wrapper sends `+1`; the reducer adds it to the running total.

```
class SQLBudgetState(AgentState):
    sql_call_count: NotRequired[Annotated[int, operator.add]]

class SQLBudgetMiddleware(AgentMiddleware):
    state_schema = SQLBudgetState
```

Three ideas make the counter work:

1. **The middleware adds a key to agent state.** `state_schema = SQLBudgetState` tells `create_agent` that the state now has `sql_call_count` alongside `messages`. `NotRequired` means it can be missing at the start; the code reads it with a default of 0.
2. **The reducer is `operator.add`.** The middleware never writes the total. It always sends `1`, and LangGraph adds that to whatever is there: 0 → 1 → 2 → 3.
3. **A `Command` carries both updates.** A plain `ToolMessage` can only add a message. To also change `sql_call_count`, `_add_one` returns `Command(update={"messages": [result], "sql_call_count": 1})`.

> **Why state and not a Python variable?** State is part of the run: it starts fresh for every `agent.invoke`, it is visible in the final result (`result["sql_call_count"]`), and it is saved with the conversation if you add a checkpointer. A counter on the middleware object would be shared by every run and every user.

7 Blocking a call

Budget = 3: the fourth run_query is answered by the wrapper

call	count before	wrapper	reaches PostgreSQL?
1st	0	run, count → 1	yes
2nd	1	run, count → 2	yes
3rd	2	run, count → 3	yes
4th	3	block, count stays 3	no

```
SQL budget reached: 3 run_query calls have already been used. Do not execute another SQL query. …
```

Figure 7. Calls 1–3 run and are counted. Call 4 gets the wrapper's own message instead.

When the budget is used up, the wrapper returns this `ToolMessage` without calling `handler`:

```
ToolMessage(
    content="SQL budget reached: 3 run_query calls have already been used. "
            "Do not execute another SQL query. Answer using the results you already have, "
            "or explain what remains unknown.",
    tool_call_id=request.tool_call["id"],
    name="run_query",
)
```

- **Why return a message instead of raising an error?** The model asked for a tool and is waiting for its answer. Every tool call must get a `ToolMessage` with the matching `tool_call_id`. A raised error would stop the whole run; a message lets the model carry on.
- **Why that wording?** It tells the model what happened, what not to do ("do not execute another SQL query"), and what to do instead (answer with what it has, or say what is unknown). It is written as instructions, because the model will read it as the next observation.
- **Blocked calls are not counted.** `_budget_message` returns a plain `ToolMessage`, not a `Command`, so the counter stays at 3.

> This was checked on the notebook's agent with a budget of 1. The model asked for `run_query` twice; the database saw exactly one query; the second request got the budget message; `sql_call_count` ended at 1.

8 Sync and async hooks

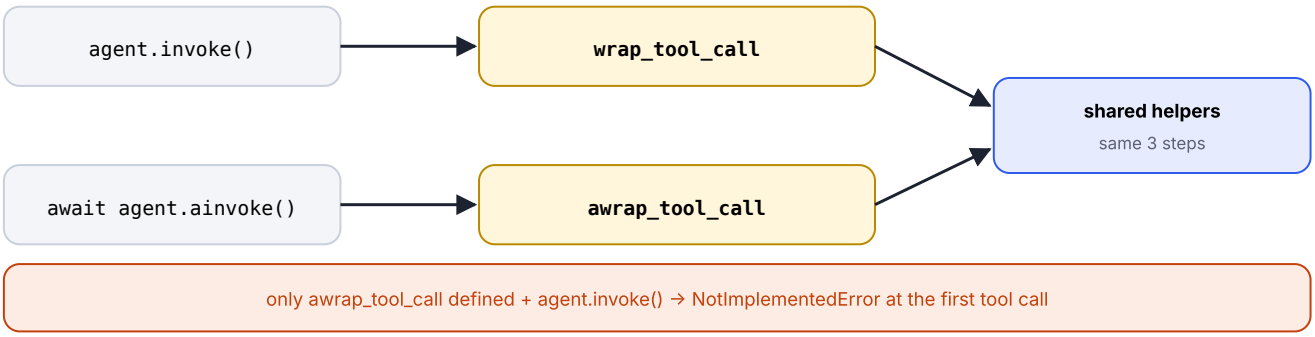


Figure 8. Each way of running the agent calls its own hook. Define both.

`agent.invoke()` and `agent.stream()` run the **sync** hook, `wrap_tool_call`. `agent.ainvoke()` and `agent.astream()` run the **async** hook, `awrap_tool_call`. The only real difference between the two is `await`:

```
def wrap_tool_call(self, request, handler):
    ...
    return self._add_one(handler(request))

async def awrap_tool_call(self, request, handler):
    ...
    return self._add_one(await handler(request))
```

A real bug from the earlier draft. The first version of this middleware defined only `awrap_tool_call`, and the notebook ran the agent with `agent.invoke()`. LangChain then raises `NotImplementedError: Synchronous implementation of wrap_tool_call is not available…` at the first tool call. Defining both hooks, with shared helpers, fixes it and keeps them in step.

9 Registering middleware

```
agent = create_agent(
    model=model,
    tools=[lookup_schema, lookup_examples, lookup_common, validate_query, run_query,
web_search],
    system_prompt=SYSTEM_PROMPT,
    middleware=[SQLBudgetMiddleware(max_sql_calls=MAX_SQL_CALLS)],
)
```

Fact	Checked on the real library
Graph nodes	still `['__start__', 'model', 'tools', '__end__']`: a tool wrapper runs *inside* the `tools` node, so it adds no node
State	gains `sql_call_count` from the middleware's `state_schema`
Order of several middleware	the first in the list is the outermost: `enter FIRST → enter SECOND → tool → exit SECOND → exit FIRST`
Same class twice	rejected: `Please remove duplicate middleware instances.`

The budget is a constructor argument, so changing it means changing one number, not editing the prompt. The notebook uses that to build a second agent with `max_sql_calls=1` to show the block (section 10).

10 One run, step by step

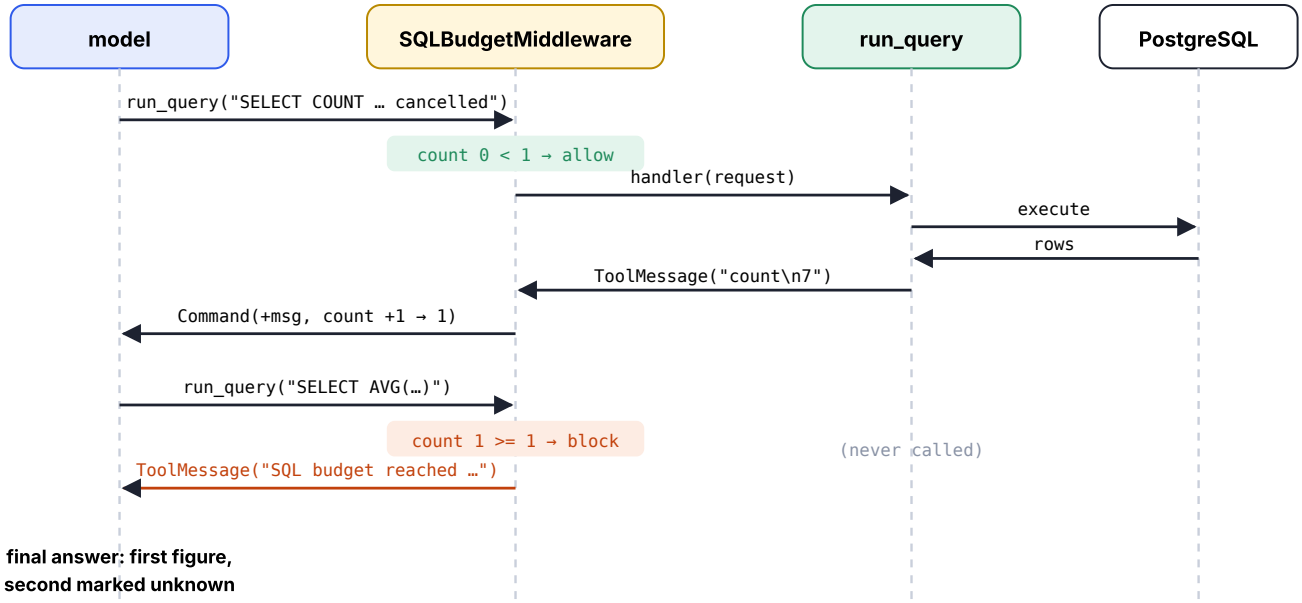


Figure 9. The notebook's `tight_agent` (budget 1) asked for two separate figures. Values are illustrative.

```
result["messages"] (run_query part only):
  AIMessage(tool_calls=[run_query(sql="SELECT COUNT(*) ... 'cancelled'")])
  ToolMessage("count\n7\n(1 rows.)", name="run_query")  ← ran, count → 1
  AIMessage(tool_calls=[run_query(sql="SELECT AVG(total) FROM orders")])
  ToolMessage("SQL budget reached: 1 run_query calls have ...",
              name="run_query")                          ← blocked, count stays 1
  AIMessage("ANSWER: 7 cancelled orders. The average order value could not be
             computed within the SQL budget.")

result["sql_call_count"]  → 1
```

`ask_agent` prints `run_query executions used: 1` at the end of the trace. In the trace, the blocked call looks like any other tool call. Only the observation tells you it was blocked.

11 Why code, not a prompt rule

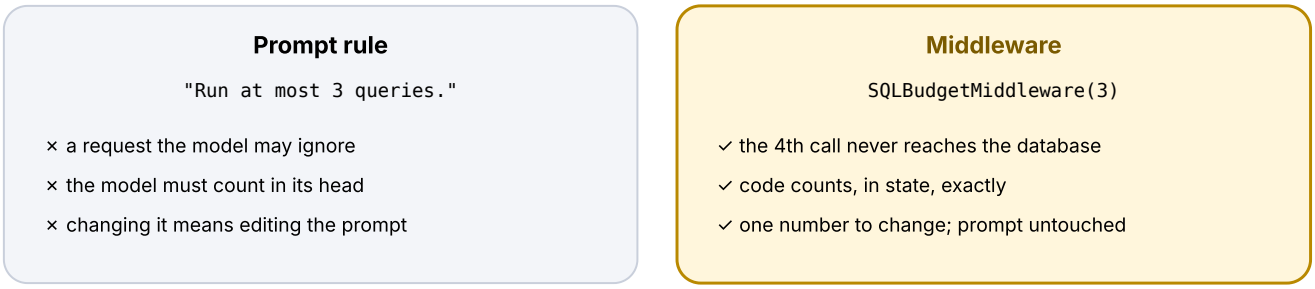


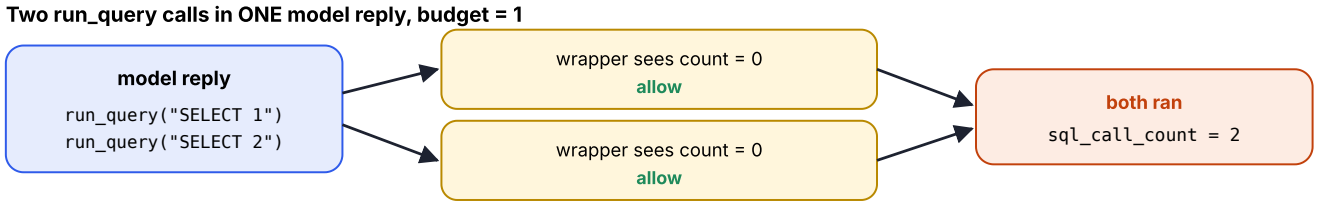
Figure 10. A prompt asks. Middleware enforces.

This is the same principle as the safety layers in Version 2 and the policy in Version 4: **rules that must hold go in code**. The prompt guides the model's choices; code decides what is actually allowed to happen.

The prompt is deliberately unchanged. The model learns about the budget only when it hits it, through the block message. That keeps this lesson clean: the only new thing is code. In a real system you may also mention the budget in the prompt, so the model plans its queries, but the middleware stays the guarantee.

> Middleware is also how Version 4's open gap could be closed. “Validate before run” is only a prompt rule there. A tool wrapper could check, before letting `run_query` through, that the same SQL passed `validate_query` earlier in the messages.

12 What the budget does not do



Both calls are handled in the same tools step. Each wrapper reads the state as it was **before** the step, so both see 0. The two +1 updates are merged afterwards. Checked on the real library: 2 queries ran, count 2.

The budget still stops the next reply: any further run_query sees 2 and is blocked.

Figure 11. The one case where the budget can be exceeded.

Situation	What happens
Lookups, `validate_query`, `web_search`	never counted, never blocked (STEP 1 passes them through)
`run_query` returns `SQL error:` or `REFUSED:`	**counted** — every attempt uses a slot, success or not
Several `run_query` calls in one model reply	all see the same starting count; the budget can be exceeded within that one step (Figure 11)
A checkpointer with a `thread_id`	state is saved between turns, so the budget covers the whole conversation, not each question
Total cost or time	not limited — only SQL executions are counted

> These limits are written into the code's “Does NOT” comment block, so anyone reading the middleware knows where its guarantee ends.

13 The limitation, and what Version 7 changes

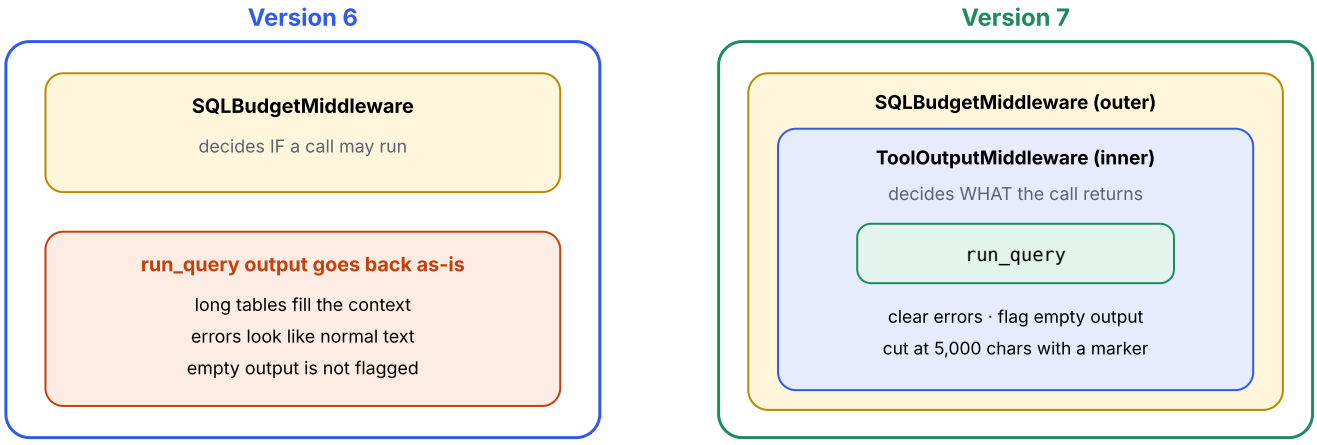


Figure 12. Version 7 adds a second tool wrapper inside the first.

The budget controls **how often** SQL runs. It does nothing about **what comes back**. `run_query` output still reaches the model unchanged:

- A 50-row result with wide columns can run to many thousands of characters, and it is resent on every later pass.
- A `SQL error:` looks like any other text unless the model reads it carefully.
- An empty or odd result is passed on without comment.

	Version 6	Version 7
Middleware	`SQLBudgetMiddleware`	+ `ToolOutputMiddleware`
Wrapper moves used	pass through · block · add to result	+ **rewrite** the result text
Order	one wrapper	budget outermost, output cleaner inside
Tools / prompt	unchanged	unchanged

14 Key takeaways

1. **Middleware adds behaviour at fixed points in the agent loop** without changing the model, the tools or the prompt.
2. **There are six hooks**: `before_agent`, `before_model`, `wrap_model_call`, `after_model`, `wrap_tool_call`, `after_agent` (plus async versions).
3. **`wrap_tool_call` sits between the model's tool request and the tool.** It runs inside the `tools` node, so the graph keeps its four nodes.
4. **A wrapper gets `request` and `handler`.** Code before `handler(request)` runs before the tool; code after runs after; not calling it means the tool never runs.
5. **Three moves:** pass through, block, change the result. The budget uses all three.
6. **The counter lives in agent state**, added by `state_schema`, merged by the `operator.add` reducer, and updated through a `Command`.
7. **Block with a message, not an error.** Every tool call needs a `ToolMessage` with its `tool_call_id`; the message tells the model what to do next.
8. **Define both `wrap_tool_call` and `awrap_tool_call`**, sharing the same helpers.
9. **First middleware in the list is the outermost.**
10. **A prompt asks; middleware enforces.** Rules that must hold belong in code.
11. **Know the edges:** errors count, other tools do not, parallel calls in one reply share a starting count, and a checkpointer makes the budget per conversation.

> **Next — Version 7:** `ToolOutputMiddleware` wraps `run_query` inside the budget and rewrites its output: clear error messages, a flag for empty results, and a 5,000-character limit with a “do not infer missing rows” marker.